

Technical Report — EV Range Prediction

Overview — Problem & Objective

The goal is to predict an electric vehicle's official driving range (range_km) from its static specification sheet — battery, performance, charging, dimensions, drivetrain, body type, seating, cargo and towing fields — without using any trip-level telemetry, since none is available in this dataset.

Target variable: **range_km** (official driving range, km).

Key restriction respected throughout: **efficiency_wh_per_km** was never used as a model input. Range, battery capacity and efficiency are algebraically linked ($\text{range} \approx \text{battery_capacity} / \text{efficiency} \times 1000$), so including it would let the model reconstruct the target instead of genuinely learning from specifications. It was used only for a training-data sanity check.

Dataset: 478 EV models, one row per model, 22 columns covering battery, performance, charging, dimensions, drivetrain, body type, seating, cargo and towing specifications.

1. Data Cleaning & Preprocessing

1.1 Data Quality Issues Identified During Profiling

- battery_type is constant (“Lithium-ion” for all 478 rows) — zero variance, no predictive value.
- fast_charge_port is almost entirely CCS (476 rows CCS, 1 row CHAdeMO, 1 missing) — near-zero variance.
- source_url is listing metadata, not a specification.
- number_of_cells is missing in 202 of 478 rows (~42.3%) — kept as a feature, but paired with a missingness flag rather than trusted blindly.
- torque_nm missing in 7 rows (~1.5%); fast_charging_power_kw_dc missing in 1 row (~0.2%); towing_capacity_kg missing in 26 rows (~5.4%) — low enough to impute inside the modelling pipeline.
- cargo_volume_l contains 3 non-numeric entries recorded as “N Banana Boxes” instead of litres: Audi Q6 e-tron quattro (10 boxes), Maxus MIFA 9 (31 boxes), Mercedes-Benz EQS SUV 580 4MATIC (13 boxes).
- model has 1 missing value.
- No exact duplicate rows were found (0 exact duplicates; 0 duplicate brand/model pairs).

1.2 Missing Value Summary

Column	Missing count	Missing %
number_of_cells	202	42.3%
towing_capacity_kg	26	5.4%

Column	Missing count	Missing %
torque_nm	7	1.5%
fast_charging_power_kw_dc	1	0.2%
fast_charge_port	1	0.2%
model	1	0.2%
cargo_volume_l	1	0.2%

1.3 Cleaning Decisions & Justification

Decision	Why
Drop exact duplicate rows and rows with an invalid/missing target	0 duplicate rows and 0 invalid targets were found, so no rows were actually removed — the check runs regardless, for safety on future data.
Treat the 3 “N Banana Boxes” entries in cargo_volume_l as missing (masked to NaN), with no assumed litre conversion	There is no reliable, documented conversion factor from banana boxes to litres; guessing one would silently fabricate data. Two flags record this instead: cargo_unknown_unit_flag (recorded in boxes) and cargo_missing_flag (litre value unavailable).
Fill missing model with “Unknown”	Identifier only, never used as a model feature.
Create cells_missing_flag before imputing number_of_cells	Lets the model learn whether “unknown cell count” itself is informative (e.g. catalogue under-reporting) instead of silently masking it.
Drop battery_type, fast_charge_port, source_url	Zero / near-zero variance or non-specification metadata — no predictive signal.
Defer imputation of remaining numeric gaps (number_of_cells, torque_nm, fast_charging_power_kw_dc, towing_capacity_kg, cargo_volume_l) into the modelling pipeline	Imputing before the train/test split would leak test-fold distribution into training; fitting the imputer on the training fold only avoids this.

After cleaning, the 3 banana-box entries push cargo_volume_l's missing count from 1 to 4 — this is expected, since those entries are deliberately treated as unknown rather than converted.

1.4 Preprocessing Pipeline

- Numeric features: median imputation (empty columns kept rather than dropped) + StandardScaler.
- Categorical features (drivetrain, segment, car_body_type): constant imputation with the label “Unknown” + one-hot encoding (unknown categories at prediction time are ignored, not rejected).
- Both are wrapped in a single scikit-learn ColumnTransformer inside the overall Pipeline, together with the feature-engineering step, and fit only on the training fold — so the identical transformation sequence replays on any new input at prediction time.

2. Exploratory Data Analysis

The train/test split is performed before EDA so that every plot and statistic below reflects the training rows only. The split groups rows by brand and the first word of the model name (a model-family proxy), then uses a grouped shuffle split (test size 20%, random_state = 42) so that closely related trims of the same model cannot appear in both the training and test sets.

2.1 Target Distribution — range_km (training rows only, n = 391)

Statistic	Value (km)
Count	391
Mean	396.25
Std. deviation	104.91
Min	135
25th percentile	320
Median (50%)	405
75th percentile	470
Max	685

range_km is roughly unimodal and right-skewed, with a long tail toward large-battery / high-performance vehicles. No statistical outlier removal was applied — these are genuine vehicles the model must generalise to; invalid targets are instead checked during cleaning (none were found).

2.2 Correlation with Target

Pearson correlation of each numeric column (including engineered missingness flags) with range_km, computed on the training fold, strongest to weakest:

Feature	Correlation with range_km
battery_capacity_kWh	0.872
top_speed_kmh	0.717
fast_charging_power_kw_dc	0.712
torque_nm	0.637
width_mm	0.492
length_mm	0.445
number_of_cells	0.323
towing_capacity_kg	0.288

Feature	Correlation with range_km
cargo_unknown_unit_flag	0.035
cells_missing_flag	-0.026
cargo_missing_flag	-0.026
efficiency_wh_per_km (excluded feature)	-0.058
cargo_volume_l	-0.089
seats	-0.296
height_mm	-0.447
acceleration_0_100_s	-0.702 (faster acceleration → higher range, since high-performance EVs tend to carry larger batteries)

battery_capacity_kWh is the strongest single numeric correlate of range, as expected physically.

efficiency_wh_per_km's raw correlation with range is weak here — the case for excluding it rests on the algebraic link to the target once battery capacity is known, not on this correlation being large.

2.3 Categorical Relationships

- Range by drivetrain (FWD / RWD / AWD) was compared via boxplot — the spread and median differ visibly enough that drivetrain was kept as a categorical feature.
- Range by car_body_type was compared across the observed body styles via boxplot — the distributions differ enough that car_body_type was kept as a categorical feature.

2.4 Efficiency Sanity Check (not a model input)

A scatter plot of range_km against efficiency_wh_per_km was produced on training data only, explicitly labelled as a sanity check. It is not used as a model input at any stage; the exclusion decision is driven by the algebraic relationship between range, battery capacity and efficiency, not by the strength of this raw correlation.

3. Feature Engineering

Beyond the raw columns, the following specification-based features were engineered inside the pipeline:

Feature	Rationale
footprint_m2	length × width — a measure of overall vehicle size / drag area
volume_m3	footprint_m2 × height — bounding-box volume, not true cabin volume
battery_per_seat	battery_capacity_kWh normalised by seats — “battery generosity” relative to vehicle purpose
torque_per_100kwh	torque_nm normalised by battery_capacity_kWh — proxy for performance-tuned vs. range-tuned drivetrains

Feature	Rationale
cells_missing_flag	whether number_of_cells was reported at all
cargo_missing_flag	whether a usable cargo volume in litres is available
cargo_unknown_unit_flag	whether cargo volume was recorded in banana boxes rather than litres

number_of_cells is retained as a numeric feature (its high missingness caps its usefulness rather than ruling it out) instead of being dropped outright. There is no segment_group roll-up feature in the notebook — segment is used as-is.

4. Model Development

4.1 Train / Test Split & Pipeline

- brand and model excluded (identifiers, not specifications a judge would enter into a tester).
- efficiency_wh_per_km excluded (target leakage, see Overview).
- Grouped 80/20 train/test split (GroupShuffleSplit, random_state = 42), grouped by brand + model-family so related trims stay together → Train: 391 rows, Test: 87 rows (15 raw specification columns each).
- After feature engineering, 19 numeric features (12 raw + 3 missingness flags + 4 engineered: footprint_m2, volume_m3, battery_per_seat, torque_per_100kwh) and 3 categorical features (drivetrain, segment, car_body_type) feed the model.

4.2 Model Comparison (5-fold Grouped Cross-Validation, training set only)

Seven candidates — a mean baseline plus six regressors — were compared with 5-fold GroupKFold cross-validation on the training set only, using the same brand/model-family groups as the train/test split so that related trims never span a fold boundary.

Model	CV RMSE mean (km)	CV RMSE std
Ridge	23.74	2.73
Linear Regression	24.92	4.14
Lasso	25.31	2.28
Gradient Boosting	31.51	6.94
Random Forest	34.95	8.03
XGBoost	36.47	10.06
Mean Baseline	104.58	12.17

Ridge and Linear Regression come out ahead in cross-validation on this dataset; the tree-based ensembles (Gradient Boosting, Random Forest, XGBoost) trail behind, likely reflecting the small size of the dataset (478 rows) relative to their capacity. A deep neural network was intentionally not tried for the same reason.

4.3 Hyperparameter Tuning (GridSearchCV, same grouped 5-fold CV)

The top two models by CV RMSE — Ridge and Linear Regression — were taken forward for tuning.

Model	Best parameters found	Best CV RMSE (km)
Ridge	alpha = 1 (grid: 0.1, 1, 5, 10, 20, 50, 100)	23.74
Linear Regression	no hyperparameters to tune	24.92

4.4 Final Model Selection

Final model selected: **Ridge Regression (alpha = 1)**. The selection rule is the lowest training cross-validation RMSE, decided before the held-out test set is touched — Ridge's 23.74 km CV RMSE was the lowest of all candidates, tuned or not. The held-out test metrics in Section 5 are reported for evaluation only; they do not feed back into model selection, which avoids overfitting the selection decision to a single test split.

5. Model Evaluation

5.1 Final Held-Out Test Set Results

The test set (87 rows) was untouched during model selection and tuning — these are genuine out-of-sample numbers, compared against a mean-value baseline:

Model	MAE (km)	RMSE (km)	R ²
Ridge (final model)	19.19	23.61	0.9375
Mean Baseline	78.55	95.97	-0.0319

5.2 Feature Importance

Computed as permutation importance on the final Ridge pipeline (5 repeats, random_state = 42): each value is the average increase in training RMSE (km) when that raw column's values are randomly shuffled, so larger numbers indicate greater reliance on that input.

Feature	Importance (Δ training RMSE, km)
battery_capacity_kWh	111.76
height_mm	43.34
car_body_type	8.97
top_speed_kmh	8.68

Feature	Importance (Δ training RMSE, km)
segment	8.64
number_of_cells	5.15
cargo_volume_l	4.83
fast_charging_power_kw_dc	4.47
drivetrain	4.37
width_mm	4.05
towing_capacity_kg	3.40
torque_nm	2.02
length_mm	0.66
acceleration_0_100_s	0.55
seats	0.32

battery_capacity_kWh dominates, consistent with domain intuition and with the correlation analysis in Section 2.2. Correlated features can share importance, and permutation importance measures reliance during training — it does not establish causality or held-out predictive contribution on its own.

5.3 Evaluation Sanity Checks

- A predicted-vs-actual scatter plot was produced on the held-out test set with a 45° reference line, to visually check for systematic bias across the range of values.
- The model comparison bar chart (Section 4.2) visualises CV RMSE with error bars ($\pm$ std across folds) for all seven candidates side by side.

6. Reproducibility

- Random seed fixed at RANDOM_SEED = 42 everywhere a stochastic step occurs (grouped train/test split, GroupKFold cross-validation, model initialisation).
- The complete pipeline (cleaning + feature engineering + preprocessing + tuned Ridge model) is saved to `ev_range_pipeline.joblib` (compress = 3).
- Reload verification: the reloaded pipeline reproduced an identical prediction to the original in-memory pipeline on a sample row (238.29 km predicted by both, vs. an actual value of 225 km for that row) — confirming the saved artifact is bit-for-bit reproducible. This check verifies serialization fidelity, not accuracy — the sample used was a training row, not a held-out test row.
- All preprocessing and feature-engineering steps are fit only on the training fold and wrapped into the same Pipeline object as the model, so the identical transformation sequence replays on any new raw specification input at prediction time.

7. Technical Documentation

This report constitutes the technical documentation of the solution, covering the problem framing, dataset, cleaning decisions, EDA findings, feature engineering, modelling pipeline, evaluation results and known limitations end-to-end, so the approach can be understood and audited without reading the source code.

7.1 Limitations

- This is a specification-based model, not a real-time range estimator: it has no access to trip-level telemetry (traffic, weather, HVAC load, State of Charge / State of Health), so it predicts the manufacturer/catalogue-listed range only.
- `number_of_cells` is missing for ~42% of rows; imputed values and the missingness flag may simply reflect catalogue reporting practices rather than a genuine engineering signal.
- Cargo entries recorded in banana boxes have no supported conversion to litres — they are treated as missing rather than estimated, which is a deliberate choice to avoid fabricating data, but does discard 3 data points' worth of cargo information.
- Model-family grouping for the train/test split is based on brand + first word of the model name; shared platforms or related vehicles marketed under different names may still occur across groups.
- With 391 training rows, performance on future EV designs well outside the specification ranges seen here (e.g. much larger batteries) is less certain.

8. Working Interactive Demonstration

An interactive website accepts a specification entry and returns a predicted range, using the saved pipeline described in Sections 4 and 6, without requiring the underlying code to be touched.

[Live demo → bayesian-blueprints-range-predictor.vercel.app](https://bayesian-blueprints-range-predictor.vercel.app)

9. Summary

Objective	Result
Final model	Ridge ($\alpha = 1$)
Test MAE	19.19 km
Test RMSE	23.61 km
Test R^2	93.75%
Leakage control	<code>efficiency_wh_per_km</code> excluded from all model inputs
Reproducibility	fixed <code>RANDOM_SEED = 42</code> ; full pipeline saved to <code>ev_range_pipeline.joblib</code>